Introduction to NLP

AIAA 4051 Lecture 14

Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

Parameter-efficent fine-tuning

- **Previous lecture**: Base foundation model → Aligned model by fine-tuning
- **This lecture**: Fine-tuning → **Parameter-efficient** fine-tuning (PEFT)

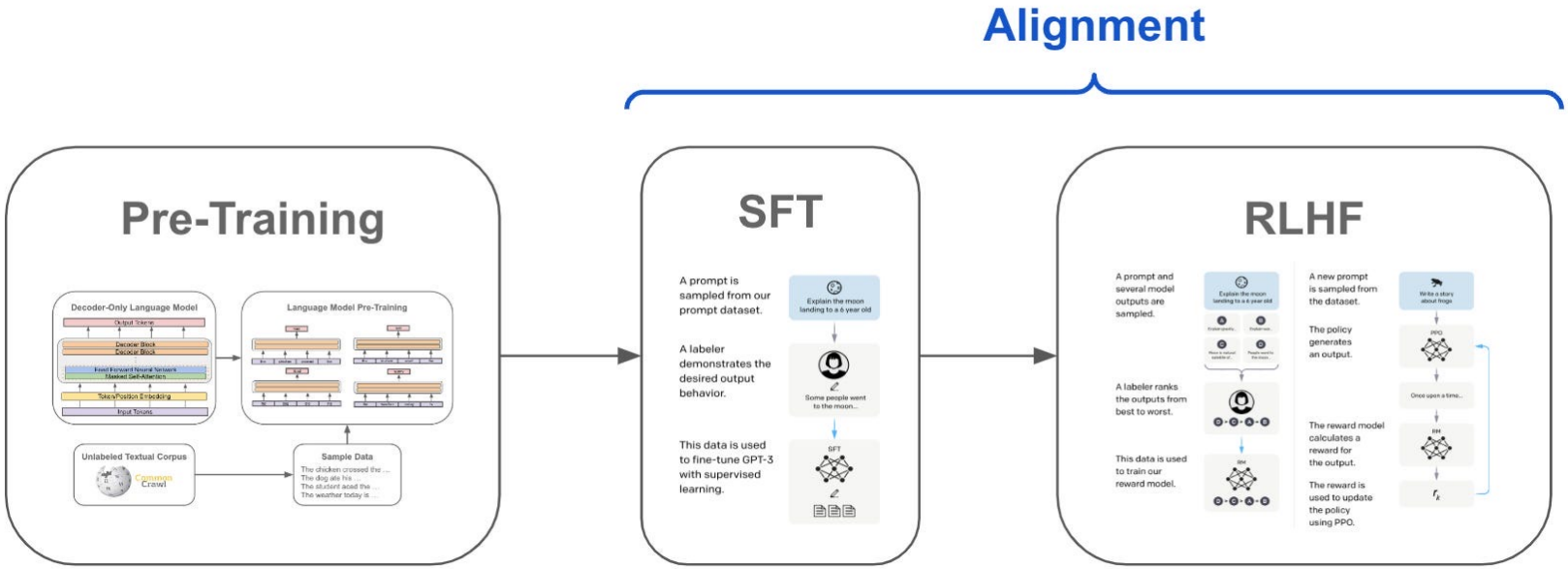
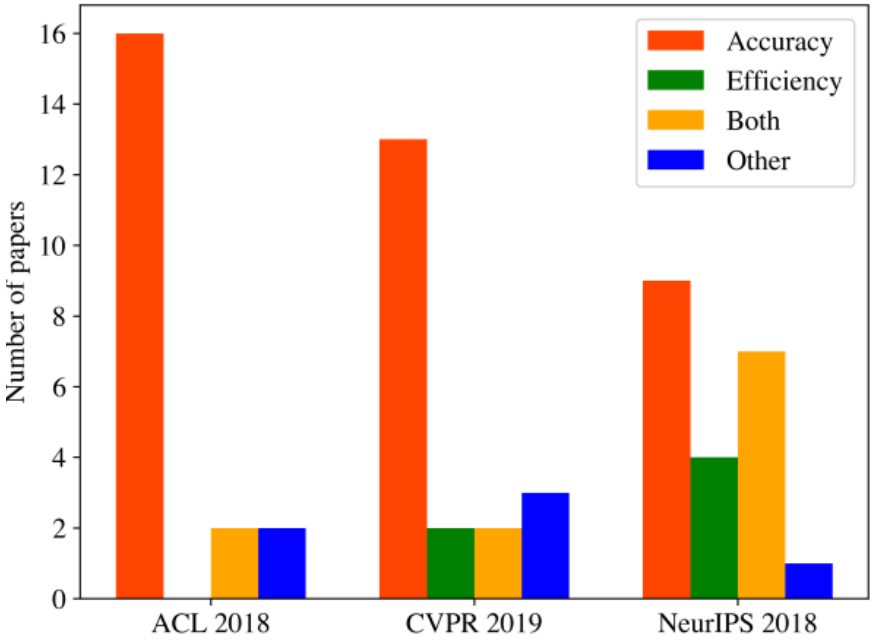


Image source: https://cameronrwolfe.substack.com/p/understanding-and-using-supervised

Start with a phenomenon

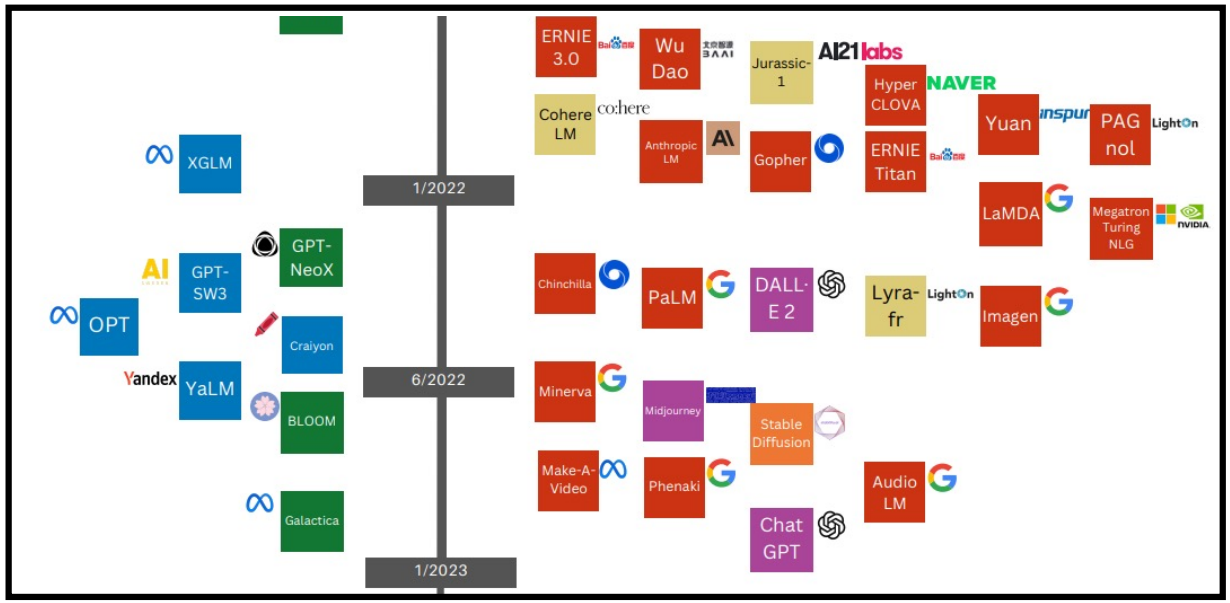
- Around 2019, the field of large models **focused more on accuracy** than efficiency.
- As training costs rise, the development of AI is becoming increasingly concentrated in **well-funded organizations**, especially in industry.

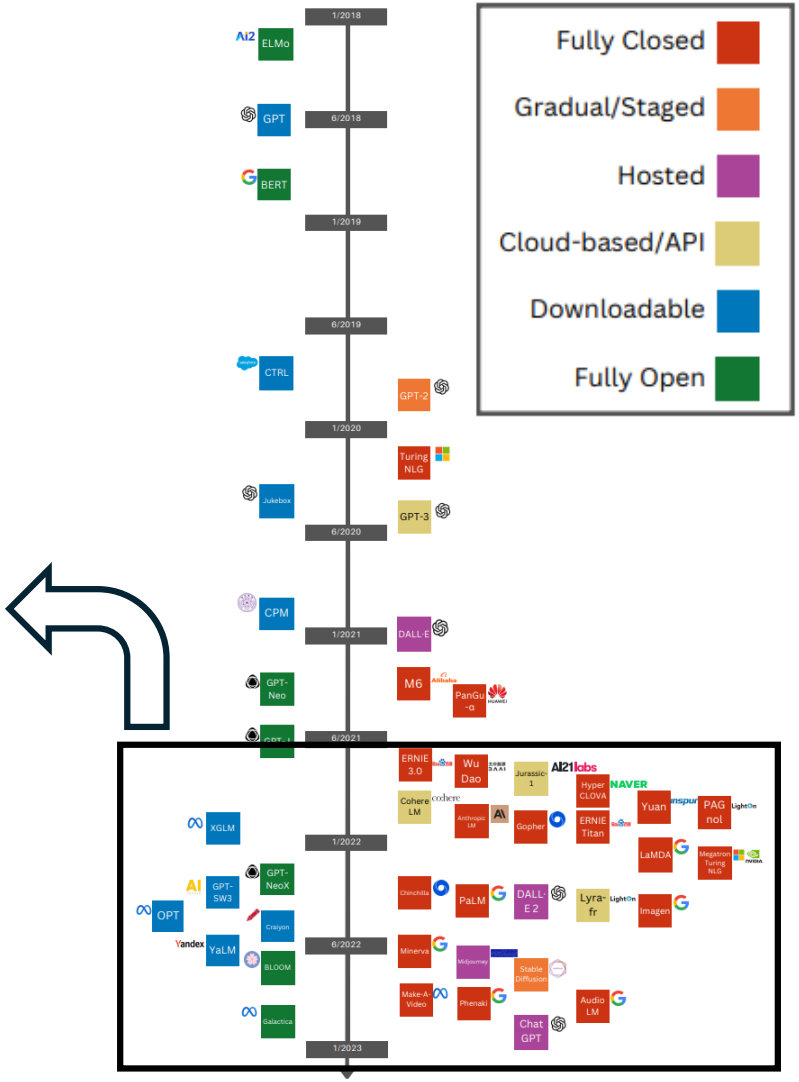


[1] Schwartz, Roy, et al. "Green ai." *Communications of the ACM* 63.12 (2020): 54-63.

Start with a phenomenon

- Much of new AI development is getting concentrated in high-resourced organizations.





Question: How can organizations with **limited computing power** train / fine-tune large models?

[2] Solaiman, Irene. "The gradient of generative AI release: Methods and considerations." Proceedings of the 2023 ACM conference on fairness, accountability, and transparency. 2023.

Fine-tuning is expensive

Full-parameter fine-tuning: **update all parameters**

E.g. 16-bit fine-tuning cost **per parameter**: (1 byte = 8 bit)

- Weight: 16 bits
- Weight Gradient: 16 bits
- Optimizer States: The Adam optimizer maintains two states.
  - First-order momentum m_t (velocity), stored in fp32.
  - Second-order momentum v_t (energy), stored in fp32.

96 bits (12 bytes) per parameter

65B model = 65 x 10^9 x 12 bytes = 780 GB GPUs = 17 A6000 (48G Per GPU)

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$$
$$v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$$
$$\hat{m}_t = m_t / (1 - \beta_1^t)$$
$$\hat{v}_t = v_t / (1 - \beta_2^t)$$
$$\theta = \theta - (\alpha * \hat{m}_t / \sqrt{(\hat{v}_t + \varepsilon)})$$

To ensure the stability of this historical data

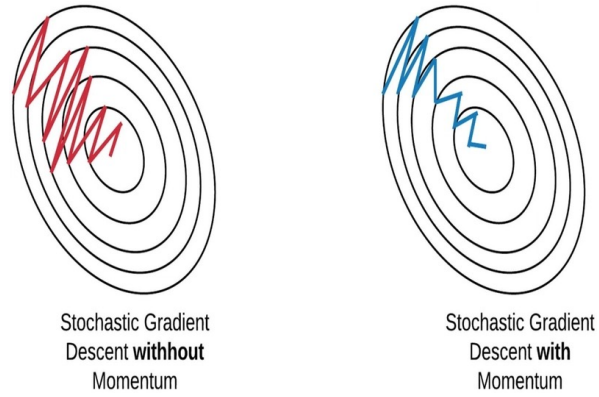


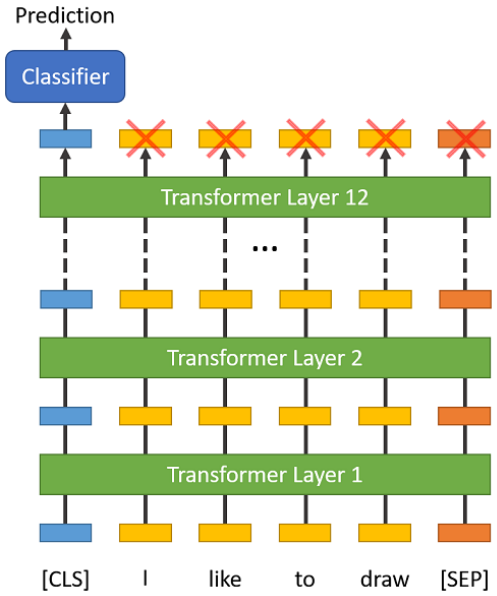
Image source: https://media2.dev.to/dynamic/image/width=800%2Cheight=%2Cfit=scale-down%2Cgravity=auto%2Cformat=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Ffmceysz6h3nvr1zo9gve.png

PEFT: Parameter-efficient Fine-tuning

Only update a small set of parameters: parameter-efficient fine-tuning (PEFT) (Instead of trying to change the entire brain, we're simply giving it "glasses.")

Fine-tuning BERT with a classifier head.

Less parameters to fine-tune, but limited adaptivity.



Fine-tuning Transformer with adapter layers at each block.

More parameters to fine-tune with more adaptivity.
Cause more latency due to added layers.

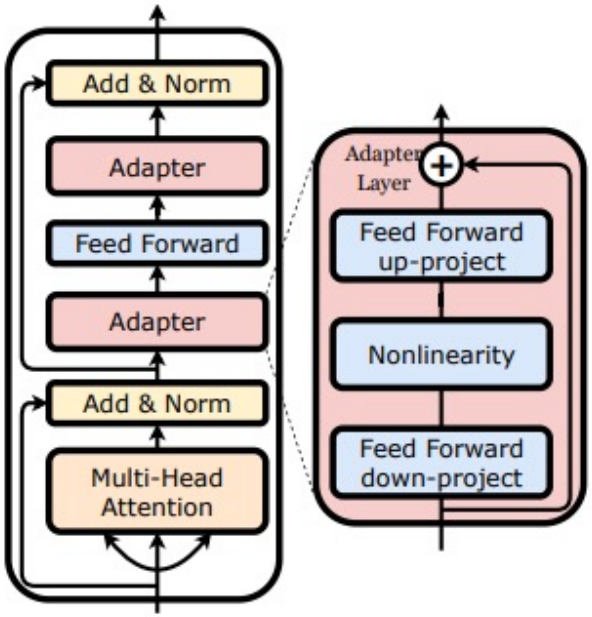


Image source:
http://www.mccormickml.com/assets/BERT/CLS_token_500x606.png

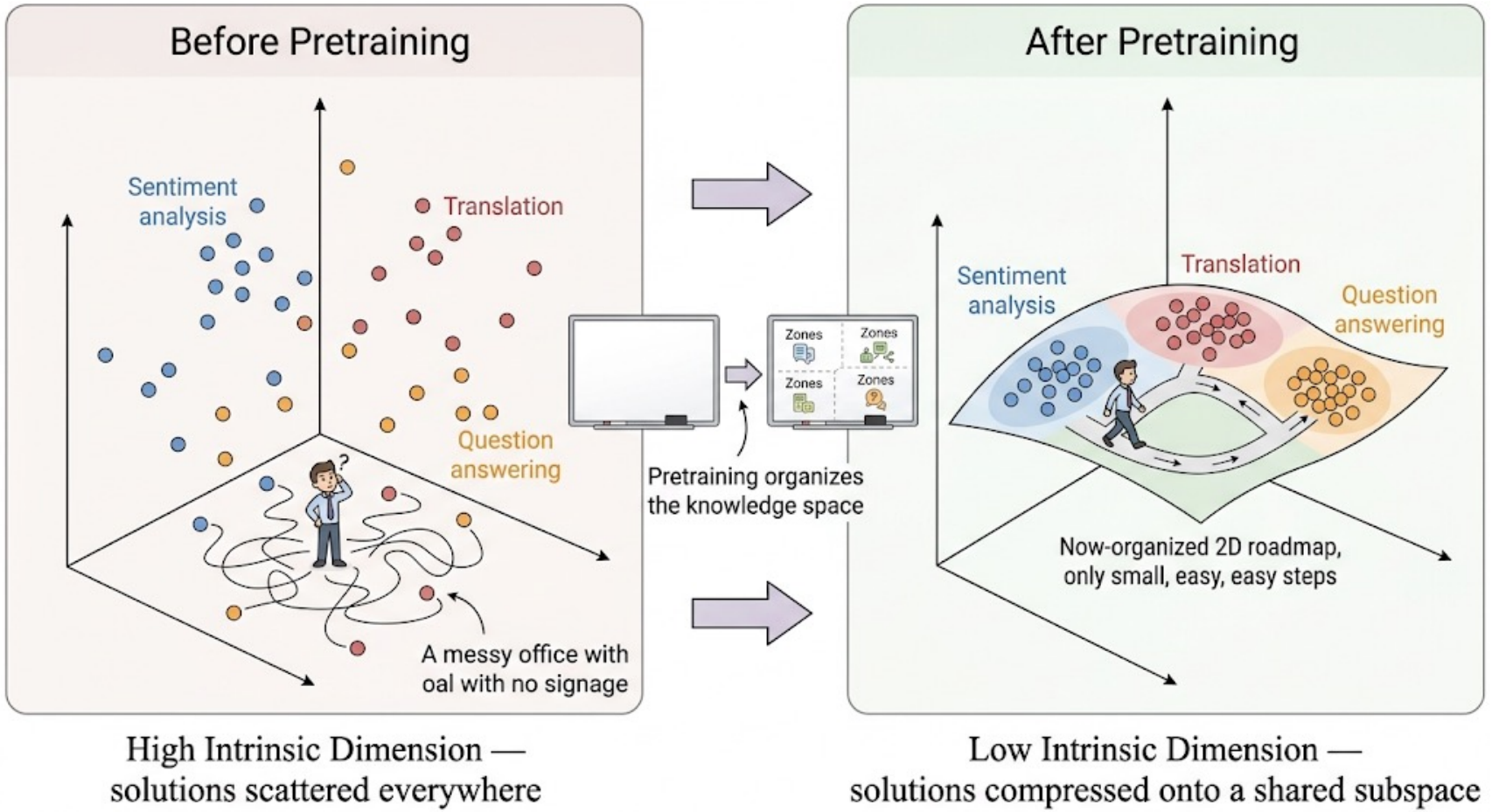
Image source:
https://miro.medium.com/v2/resize:fit:628/1*Xw3QjWiGU_ZGNTBCrAWCog.png

Questions about PEFT

Why fine-tuning a smaller number of parameters is sufficient?

- **Parameter Overload**: LLM have hundreds of billions of parameters, but when handling specific tasks, not all parameters need to participate in the changes.
- **Intrinsic Dimension:** Although the model parameter space is large, the "effective directions of change" required to solve a specific task are all in a very low-dimensional subspace.

Intuition of pre-training



Essentially, there are only a small space to search when solving a problem, after fine-tuning.

In other words, pre-training finds a good organization (representation) of concepts to make fine-tuning easier to optimize.

Intrinsic Dimension & Manifold Learning

Illusion of High Dimensions: In robotics or autonomous driving, a camera captures millions of pixels per frame (High-dimensional space).

The Reality: The true state is just a few variables in a **lower-dimensional space**, like joint angles, speed, and X, Y, Z coordinates.

Manifold Learning: Algorithms learn to unroll and discover a low dimensional space.

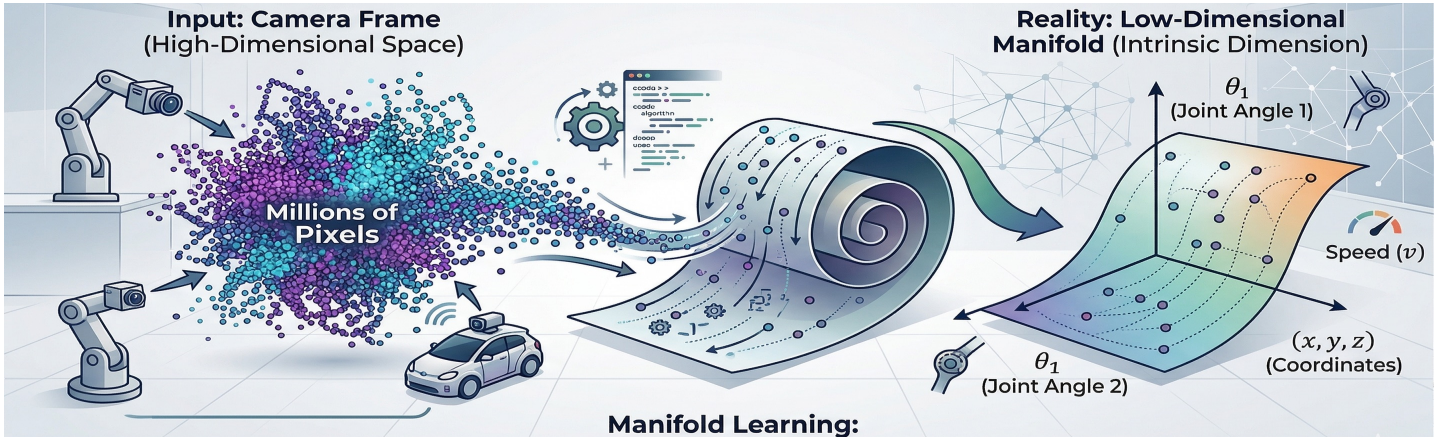


Image source: Gemini generated

Parameter Overload & Matrix Factorization

The Problem: A User-Item interaction matrix (e.g., Netflix) is huge. Many parameters: millions of users multiplied with millions of movies => a big matrix.

The Solution: We don't need this big matrix. Users and items only have a **limited number of latent properties** (e.g., genre, director, mood).

Mathematical Elegance: A massive sparse matrix R can be approximated by multiplying two much smaller matrices:

$$R \approx U \times V^T$$

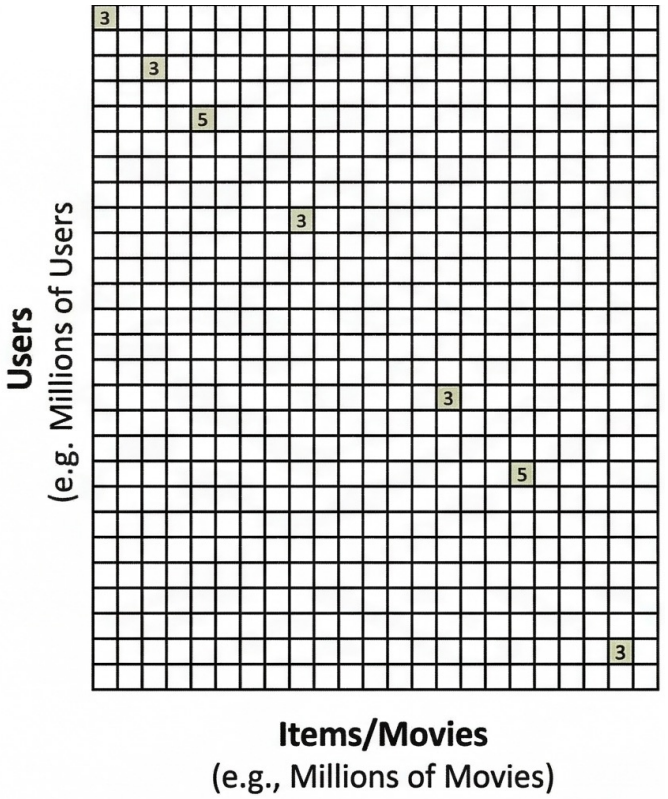
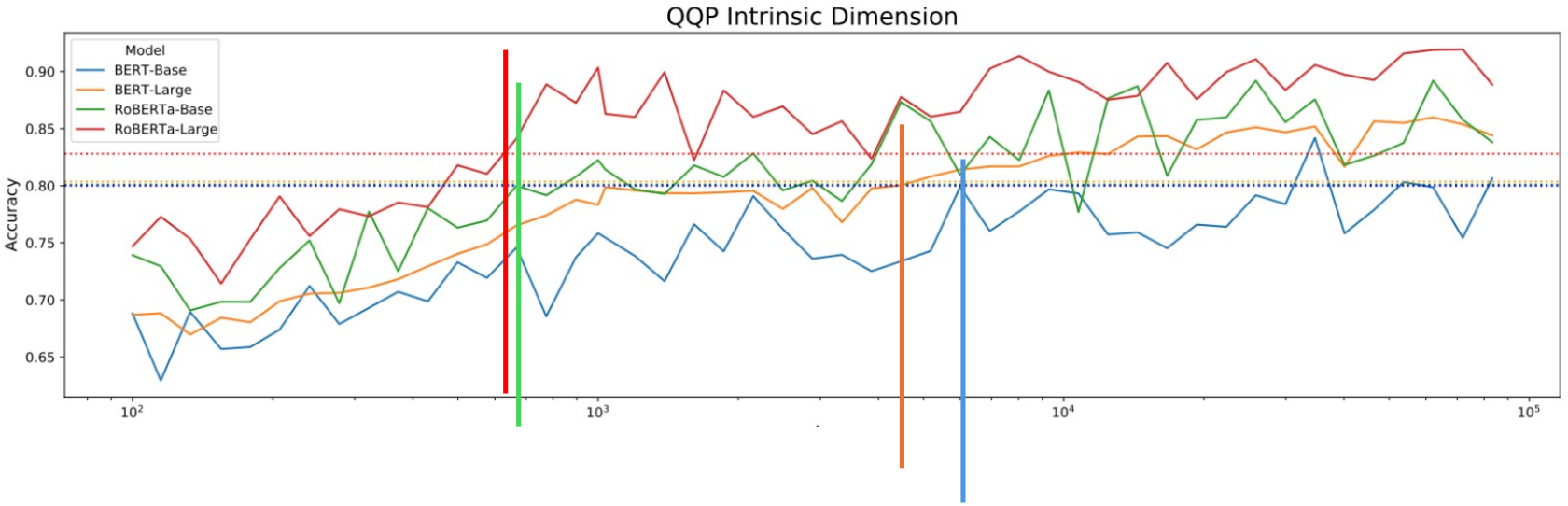


Image source: Gemini generated

Intrinsic Dimension of LLM

How many dimensions are enough?

QQP is a binary classification task for predicting semantic equality of two questions



Dashed lines are the 90% of the performance of the full-SFT models. The smallest dimension (d) to achieve 90% of the full SFT performance is defined as intrinsic dimension of a model on a task.

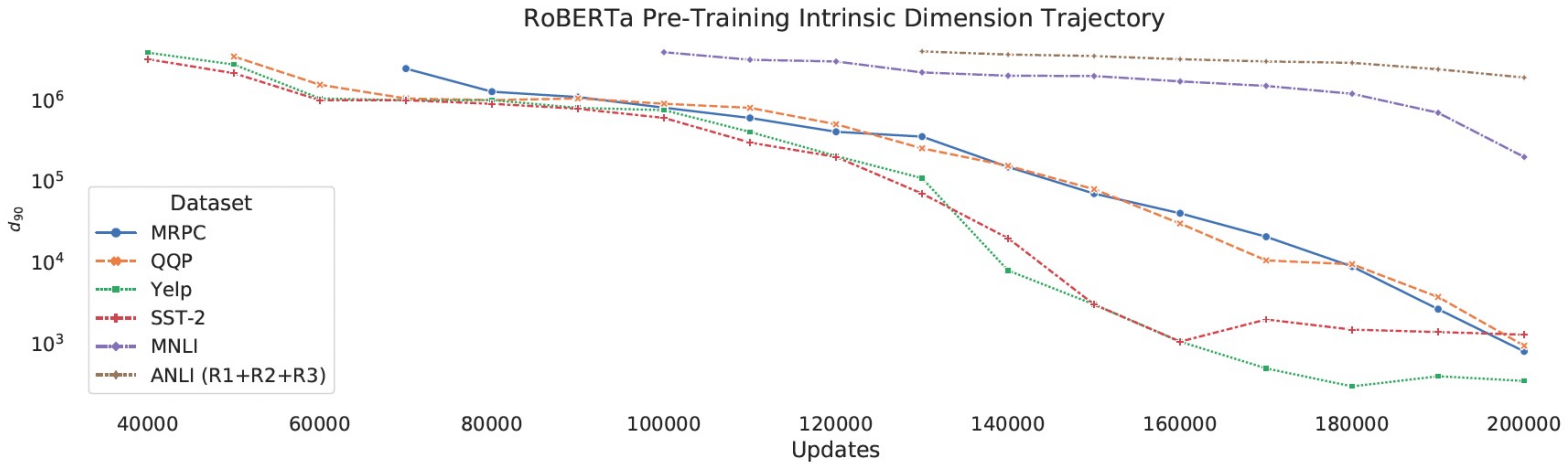
Intrinsic Dimension: When fine-tuning, parameter updates are forcibly restricted to dimension d (horizontal axis): $\theta^D = \theta_0^D + \theta^d M \qquad M = HG\Pi HB$

M is a fixed random projection matrix from a lower d-dim to a highehr D-dim.

Once d reaches a certain value, adding more dimensions will not improve the effect.

[3] Aghajanyan, Armen, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic dimensionality explains the effectiveness of language model fine-tuning." 2021.

Intrinsic Dimension



- For the same RoBERTa model, different tasks require different intrinsic dimensions.
- With more pre-training updates, the intrinsic dimensions reduces.
  - Pre-training setting the task representation at a lower-dimensional space for easy of learning.

[3] Aghajanyan, Armen, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic dimensionality explains the effectiveness of language model fine-tuning." 2021.

Intrinsic Dimension

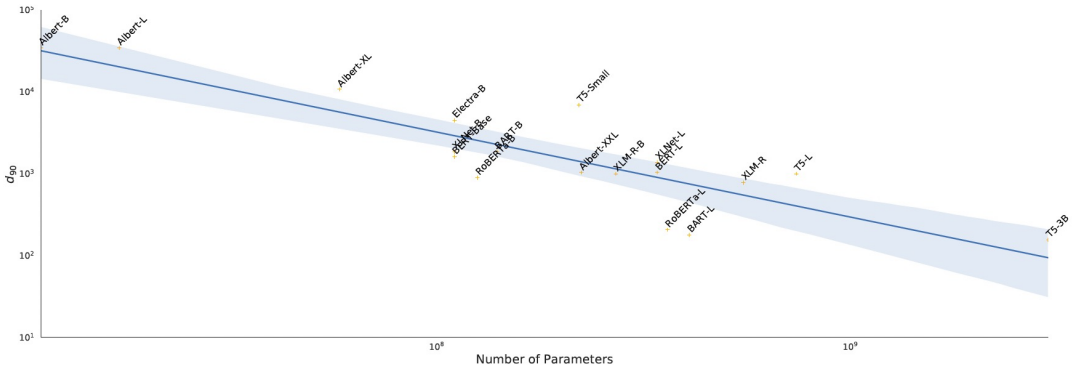


Figure 3: We calculate the intrinsic dimension for a large set of pre-trained models using the SAID method on the MRPC dataset.

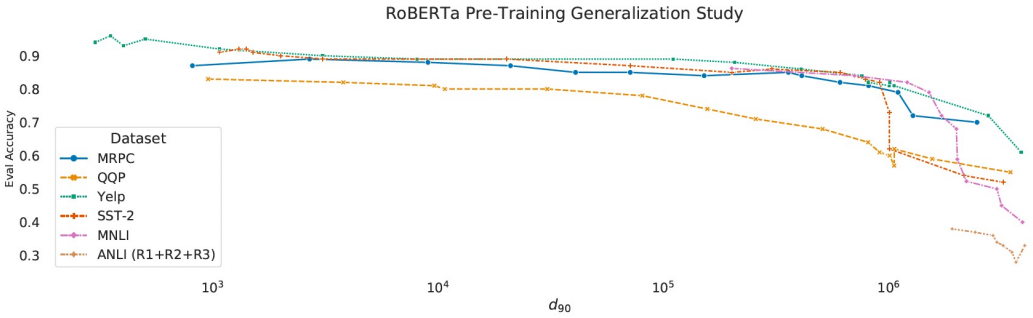


Figure 4: The evaluation accuracy of six datasets across various intrinsic dimensionalities. There is a strong general trend that pre-trained models that are able to attain lower intrinsic dimensions generalize better.

With more and more parameters in a model, the intrinsic dimension reduces.

With a higher intrinsic dimension, performance gets worse across multiple prediction tasks.

[3] Aghajanyan, Armen, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic dimensionality explains the effectiveness of language model fine-tuning." 2021.

Motivation of LoRA

Low Intrinsic Dimension:

Pre-trained models are over-parameterized. The "effective" variation during adaptation resides in a low-dimensional subspace.

Mathematical Interpretation-- Low Rank:

This implies that the weight update matrix ΔW, despite having full shape $(d \times k)$, has a low rank r $(r \ll \min(d, k))$.

$$W = \underbrace{W_0}_{d \times k} + \underbrace{\Delta W}_{d \times k}$$

Origin Param

FT Param Change

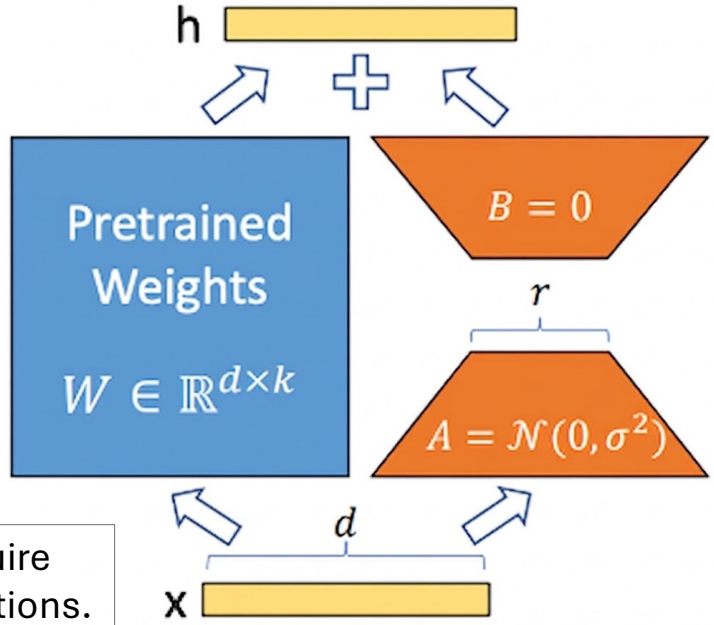
LoRA

LoRA: Low-Rank Adaptation

Instead of training ΔW directly, LoRA decomposes it into **two low-rank matrices:**

$$W = \underbrace{W_0}_{d\times k} + \underbrace{\Delta W}_{d\times k} = \underbrace{W_0}_{d\times k} + \underbrace{B}_{d\times r}\underbrace{A}_{r\times k}$$

$$h = W_0 x + \Delta W x = W_0 x + BAx$$



The attention layers require matrix-vector multiplications.

- **Modular & Switchable:** Enables sharing one frozen backbone across multiple tasks by simply swapping small adapter modules.

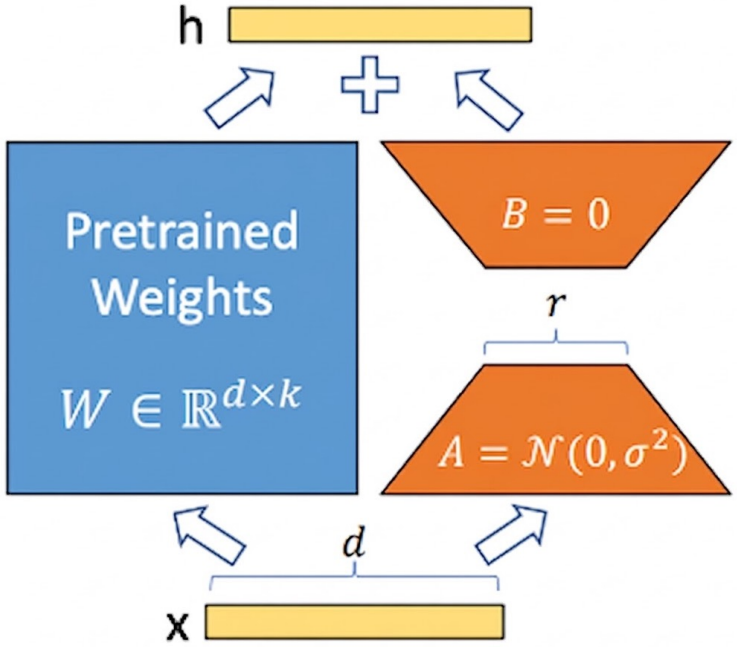
- **No Inference Latency:** Adapter weights can be merged into the base model during deployment, ensuring zero overhead.

[4] Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." ICLR 1.2 (2022): 3.

LoRa Initialization

- **Matrix A: Gaussian Initialization:** Initialized with random noise $\mathcal{N}(0, \sigma^2)$ to **break symmetry**. If all values are set to 0, the gradient updates of all neurons will be exactly the same, causing the network to be unable to learn diverse features.
- **Matrix B: Zero Initialization**: Initialized strictly with zeros so that the initial product $B \times A$ is exactly zero.

  Combined Effect: At Step 0, $\Delta W = BA = 0$
  The forward pass is $h = W_0 x + 0$
  The model starts exactly from the original pre-trained model.
  No random noise degrades the performance at the beginning.



[4] Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." ICLR 1.2 (2022): 3.

LoRA Cost

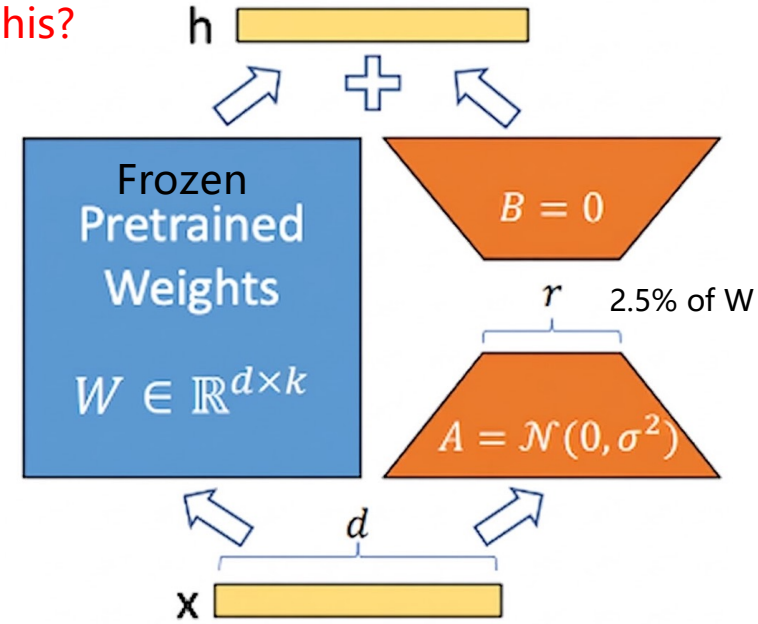
How to solve this?

E.g. 16-bit fine-tuning cost **per parameter**:

- Weight: 16 bits
- Weight Gradient: 0.4 bits
- Optimizer States: 0.8 bits
- Adapter Weights: 0.4 bits

Still Large: We still need 16 bits to store them for inference

Small: Only 2.5% of the parameters are trained

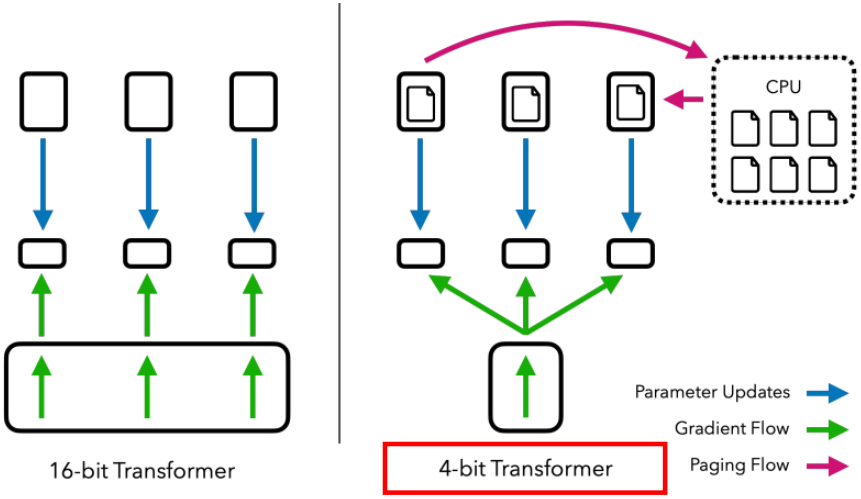


17.6 bits per parameter

65B model = 143 GB GPUs = 4 A6000 (48G Per GPU)

[4] Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." ICLR 1.2 (2022): 3.

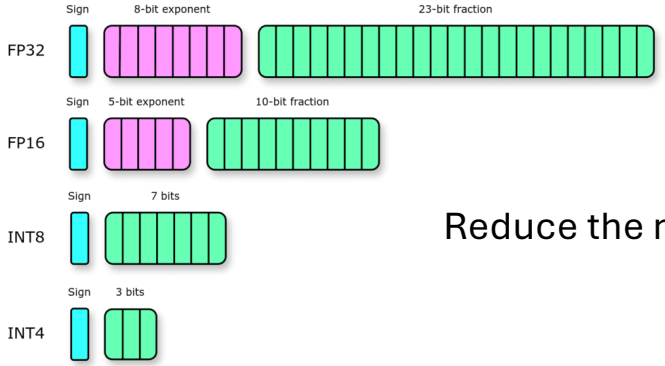
From LoRa to QLoRa



Quantize the transformer model to 4-bit precision while almost preserving the model's intelligence.

The Flaw of Standard INT4: Standard 4-bit integer (INT4) quantization spaces its 16 levels evenly.

$$\mathbf{X}^{\text{Int8}} = \text{round}\left(\frac{127}{\text{absmax}(\mathbf{X}^{\text{FP32}})}\mathbf{X}^{\text{FP32}}\right) = \text{round}(c^{\text{FP32}} \cdot \mathbf{X}^{\text{FP32}})$$



Reduce the model size

[5] Dettmers, Tim, et al. "Qlora: Efficient finetuning of quantized llms." Advances in neural information processing systems 36 (2023): 10088-10115.
Image source: https://developer-blogs.nvidia.com/wp-content/uploads/2021/07/qat-training-precision.png

Why NF4 is Optimal

- **The Nature of LLM Weights:** Pre-trained neural network weights are not randomly scattered; they follow a zero-mean **Gaussian distribution**.
  - Even quantization leads to high error for weights in the center, while wasting bits for rare extreme values.
- **The NF4 Innovation**: QLoRA introduces the NormalFloat (NF4) data type. It calculates the exact quantiles of a standard normal distribution. The intervals between levels are dynamically adjusted: extremely narrow in the middle and wide at the tails. This guarantees that exactly 1/16 of the weights fall into each of the 16 levels.
- Quantization error is roughly the expectation

  Σ_v probability(v) x error(v,quan(v))

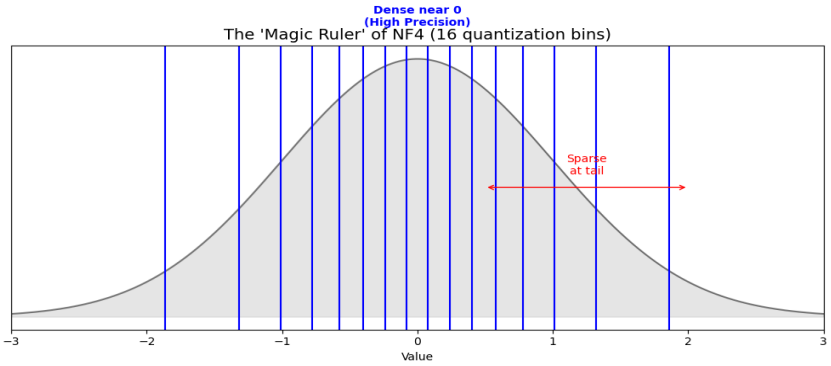
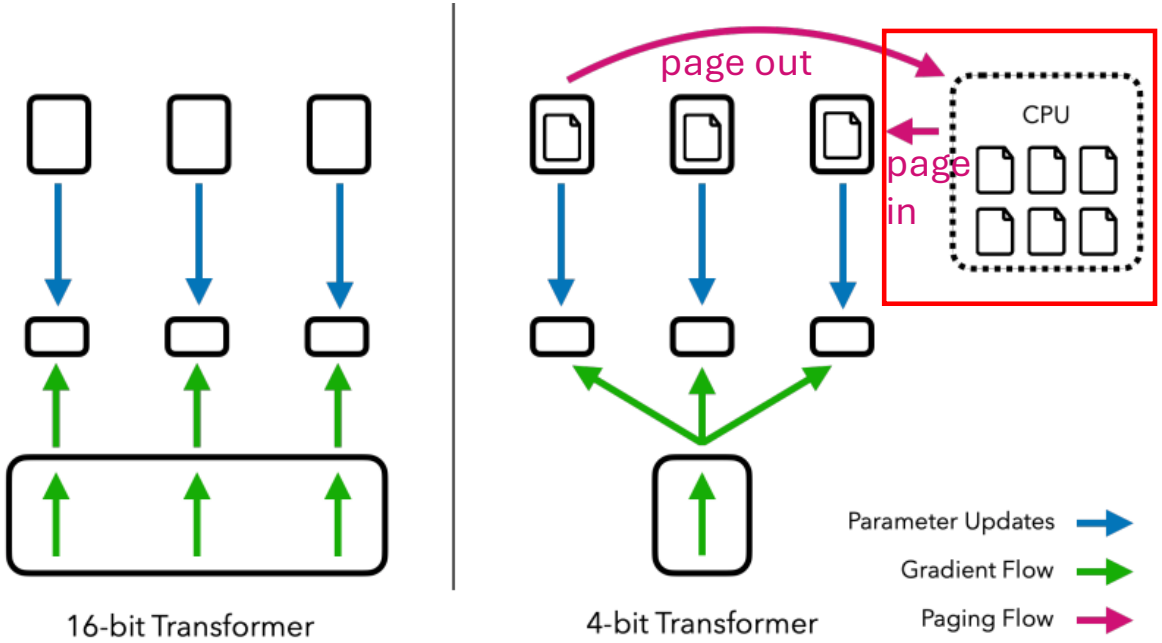


Image source: https://velog.io.....

From LoRa to QLoRa



Use paged optimizer to handle memory spikes: when the GPU's memory is insufficient (long sequence or gradients and activations are on GPU), **it will temporarily move the optimizer states to the CPU's memory (RAM)** and move them back when the GPU has free time.

A transparent mechanism: CUDA manages the page in/out when needed, designers do not need to worry about it.

Why paging in/out the optimizer states: used only when gradients are available, while they takes 64/96 of the memory.

[5] Dettmers, Tim, et al. "Qlora: Efficient finetuning of quantized llms." Advances in neural information processing systems 36 (2023): 10088-10115.

QLoRa Cost

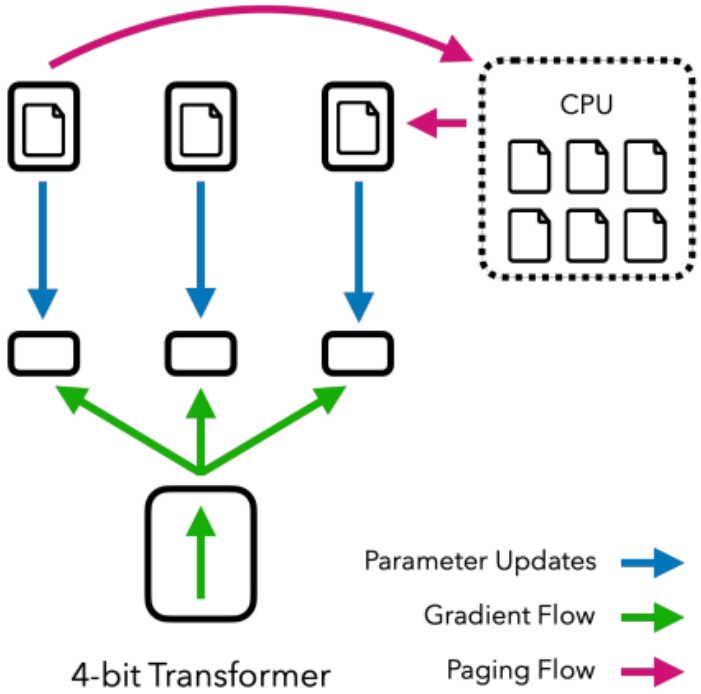
Fine-tuning with **QLoRA**

E.g. 16-bit fine-tuning cost **per parameter**:

- Weight: 4 bits — Small Now
- Weight Gradient: 0.4 bits
- Optimizer States: 0.8 bits
- Adapter Weights: 0.4 bits

5.6 bits per parameter

65B model = 45.5 GB GPUs = 1 A6000 (48G Per GPU)



LoRa in Industry

Scenario: A SaaS platform serves 1,000 clients, each needing a customized LLM.

Industry Solution: Dynamic Adapter Loading

Only a base model is persistently loaded into GPU.

When a request arrives, the system dynamically swaps in the client's specific, ultra-lightweight LoRA Adapter (~10-50M).

Example: [lorax]

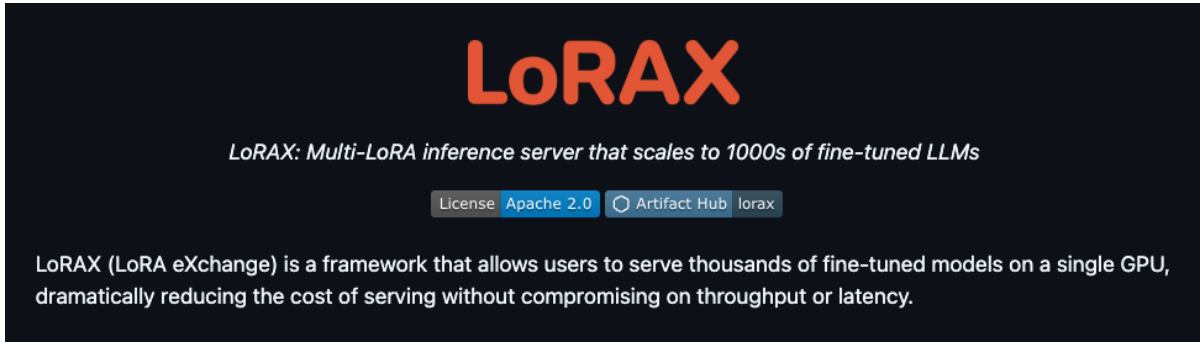


Image source: https://github.com/predibase/lorax

Demon

- Take a look at the open-sourced codes for LoRA and QLoRA in the official Huggingface library
  - https://github.com/huggingface/peft

Extra readings

- For more theoretical treatment of low-dimensional representation and compression, refer to the textbook
  - High-Dimensional Data Analysis with Low-Dimensional Models: Principles, Computation, and Applications. John Wright and Yi Ma (https://book-wright-ma.github.io/)
  - Principles and Practice of Deep Representation Learning (A Mathematical Theory of Memory). Sam Buchanan · Druv Pai · Peng Wang · Yi Ma (https://ma-lab-berkeley.github.io/deep-representation-learning-book/)

- For LLM quantization, see the papers
  - https://github.com/pprp/Awesome-LLM-Quantization
  - A Survey on Model Compression for Large Language Models. Xunyu Zhu, Jian Li, Yong Liu, Can Ma, Weiping Wang (https://aclanthology.org/2024.tacl-1.85/)

Quiz

1. Pen and paper; no compute or cellphone allowed.
2. Turn in your answer sheet when you leave the classroom.

- Q1 (T/F): In the regular SFT, the model parameters and their gradients takes the most memory space.
- Q2 (short answer): Please quantize the vector [-1.1, 0, 2.01] to their nearest integers in the range of {-1,0,1}.
- Q3 (multi-choice): what are the techniques used in QLoRA? (A) a page exchange mechanism between GPU/CPU memory; (B) highly efficient way of calculating gradient of LLM fine-tuning loss function; (C) a even-spaced quantization of model parameters; (D) compress multiple adapters, one for an NLP task, in GPU memory.
- A: F; [-1,0,1]; (A)